

Project Proposal: Solving Fluid Dynamics with Physics-Informed Neural Networks

Problem Background & Motivation

In the first five weeks of this course, we have learned a lot about standard deep learning models like CNNs and Transformers. We saw how good they are at finding patterns in data. However, we noticed a problem: these models don't actually "know" any physics. If we try to use a normal neural network to predict something physical, like how water flows or how heat spreads, it might give us an answer that looks okay but completely violates the laws of physics.

Looking at the syllabus, we saw that Part 2 of the course is about "Machine Learning for Physical Applications." We got really curious about how we can force a neural network to obey physical rules. We decided to read ahead a bit and found out about Physics-Informed Neural Networks (PINNs).

Our motivation for this project is to get a head start on this exciting topic. We want to see if we can use a neural network to simulate fluid dynamics (specifically, the Navier-Stokes equations). Normally, traditional numerical solvers can be computationally expensive. We want to find out if PINNs can do it faster and easier, and we want to learn how to actually write the code for it before we learn the deep theory in class.

Related Work

When we were looking for ideas, we found a lot of papers. We tried to organize the research we found into two main themes that make sense to us right now:

Theme 1: Standard Deep Learning vs. Physics-Based Models Most of the models we learned so far (like basic feedforward networks) just try to make the output match the training data. But for physical problems, getting enough training data from real experiments is too expensive. We saw that researchers are trying different things to fix this. For example, Neural ODEs try to model continuous time. But PINNs seem more direct to us because they just add the physics equation straight into the loss function. This means the model gets penalized if it breaks the laws of physics, which we think is a really clever idea.

Theme 2: Making PINNs Actually Run Fast We read the original paper that introduced PINNs (Raissi et al., 2019). It sounded great, but earlier implementations were reported to be slow in practice. Then, we found a newer open-source project called `pinns-torch` (Akbarian & Raissi, 2023). This project rewrote everything in PyTorch (which we are more comfortable with) and used something called CUDA Graphs to make it run much faster. This makes it possible for us to actually do this project without needing a massive supercomputer.

High-level Methodology

Our main goal is to use the `pinns-torch` library to solve a fluid dynamics problem and see how it works. Here is our plan:

- 1. Setting up the Problem:** Instead of trying to invent a new physics problem, we will use

the classic Navier-Stokes example provided in the `pinns-torch` repository. The cool thing about PINNs is that we don't need to download a huge dataset. The library has a `MeshSampler` that will automatically generate random points (collocation points) in our simulation area to train the network.

2. Training the Model: We will train a Fully Connected Network. The inputs will be the coordinates (x, y) and time (t), and it will try to predict the fluid velocity and pressure. We will let the library calculate the PDE loss (to make sure it follows Navier-Stokes) and the boundary loss.

3. Our Experiment: Once we get the baseline code running and can see the fluid visualization, we want to do some experiments. We plan to change some simple parameters in the configuration file, like the number of training points (`n_train`) or the number of layers in the network. We want to record how these changes affect the training time and the final error. We want to see if we can make it train faster without losing too much accuracy.

Fallback Plan: We are a bit worried that the Navier-Stokes equations might still be too heavy for our computers to train in time for the final deadline. If we get stuck and it takes too long, we will switch to a simpler 1D problem, like the Schrodinger equation. The `pinns-torch` library has a simple Jupyter Notebook tutorial for this, so we know we can definitely get it working as a backup plan.